
Sistemas Operacionais II

Entrega 3

Anderson Simioni Assunção (23150566)

Bruno Pavani Bertolino (23100464)

Gustavo Fukushima de Salles (21250090)

Julia Fischer Gazolla (23250586)

Classes modificadas para P3

- Ajuste do busybox - não precisa ser recompilado a cada vez
- Os componentes recebem número de porta específico

```
src >  car.cpp >  start()
```

```
1  #include "car.h"
2
3  Car::Car()
4  {
5      this->components.push_back(new PowertrainComponent<NIC>(80));
6      // each component receive a different port
7  }
8
```

Classes modificadas para P3

- A sincronização das VMs passa a acontecer em protocol, mas o mecanismo de sincronização segue igual
 - Todas enviam READY. A primeira VM que receber todos READY, envia GO via Ethernet
 - Não criamos novas classes
-

Exemplo de envio e recebimento

```
creating reader thread..
read msg thread running..
NIC sending through ETHERNET
[SYNC] Received GO, All VMs synced, Starting ticks.
[CAR COMPONENT][Gateway] initialized! Ready to start
[CAR COMPONENT][Powertrain] initializing..
initializing communicator..
creating/opening shared memory region..
mapping shared memory region..
creating reader thread..
read msg thread running..
[CAR COMPONENT][Powertrain] initialized!
[RECV t=1759281325748453 id=0]

[LATENCY t=-467389]

[RECV t=1759281325780085 id=0]

[LATENCY t=-483345]

[RECV t=1759281325965719 id=0]

[LATENCY t=-514940]

[RECV t=1759281325985898 id=0]

[LATENCY t=-121993]

[SEND t=1759281326564428 id=0]

NIC sending through ETHERNET
[RECV t=1759281326575389 id=0]

[LATENCY t=7868]

[RECV t=1759281326764024 id=1]

[LATENCY t=-471541]

[RECV t=1759281326787197 id=1]

[LATENCY t=-484408]
```

Sinal da Kernel (SIGIO)

```
130 // SIGIO
131 void EngineEthernet::sigio_handler(int) {
132     //Async-signal-safe: just post the semaphore
133     sem_post(&packet_sem);
134 }
135
136 // Receive loop
137 void EngineEthernet::rx_loop() {
138     while (running) {
139         // This blocks until the semaphore is posted by SIGIO
140         sem_wait(&packet_sem);
141
142         if (!running) break;
143
144         on_packet(); // Drain all packets in the socket buffer
145     }
146 }
147
```

Sinal da Kernel (SIGIO)

```
148 // Drain all available frames and forward to NIC
149 void EngineEthernet::on_packet()
150 {
151     if(sock_ < 0) return;
152
153     while(true)
154     {
155         uint8_t buf[ETH_FRAME_LEN]; // ~1514 bytes
156         ssize_t n = ::recvfrom(sock_, buf, sizeof(buf), 0, nullptr, nullptr);
157         if(n < 0) break; //EAGAIN when drained
158
159         if(static_cast<size_t>(n) < sizeof(ether_header)) continue;
```

Dados de latência (Ethernet)

Message #	Latency		
1	16783		Average:
2	2941		276441.55
3	400856		
4	341558		
5	615601		Median:
6	16666		340971
7	1493		
8	401956		
9	340243		Standard deviation:
10	622273		242157.1386
11	4185		
12	18476		
13	400099		
14	341273		
15	620748		
16	7089		
17	18047		
18	398437		
19	340669		
20	619438		

Conclusões

- De forma geral, a equipe conseguiu concluir as tarefas propostas para a terceira entrega do projeto de Comunicação em Sistemas Autônomos Críticos.
 - Foi possível implementar a troca de mensagens entre diferentes veículos, avaliar latência de comunicação e concluir os entregáveis para essa etapa.
 - Nas próximas etapas espera-se:
 - Mudar para x86
 - Implementar algoritmo para análise de latência
-